

Data Structures



Lecture 13 Recursion (Advanced)

Prantik Paul [PNP]

Lecturer

Department of Computer Science and Engineering

BRAC University

Recursive Definition (Binary Search)

```
if the array is empty (if  $l > r$  that is):  
    return false  
else:  
    Find the position of the middle element:  $mid = (l + r) / 2$   
    If  $key == data[mid]$ , then return true  
    If  $key > data[mid]$ , the search the right half  $data[mid+1..r]$   
    If  $key < data[mid]$ , the search the left half  $data[l..mid-1]$ 
```

Recursive Definition (Binary Search)

```
contains(a,l,r,k) =  $\begin{cases} \text{false} & \text{if } l > r \\ \text{true} & \text{if } k = a[\text{mid}] \\ \text{contains}(a,\text{mid}+1,r,k) & \text{if } k > a[\text{mid}] \\ \text{contains}(a,l,\text{mid}-1,k) & \text{if } k < a[\text{mid}] \end{cases}$ 
```

Recursive Programming (Binary Search)

```
def contains(arr, left, right, key):  
    if left > right:  
        return False    #base case  
    else:  
        mid=(left+right)//2  
        if key==arr[mid]:  
            return True    #base case  
        elif key > arr[mid]:  
            return contains(arr, mid+1, right, key) #recursive part  
        else:  
            return contains(arr, left, mid-1, key)  #recursive part
```

Recursive Programming (Find Maximum - Binary)

```
def maximum(a,b):  
    return a if a>=b else b  
  
def findMax(arr, left, right):  
    if left == right:  
        return arr[left]    #base case  
    else:  
        mid = (left+right)//2  
        maxLeftHalf=findMax(arr, left, mid)    #recursive part  
        maxRightHalf=findMax(arr, mid+1, right) #recursive part  
        return maximum(maxLeftHalf, maxRightHalf)
```

Recursive Definition (Exponentiation Efficient)

$$a^n = \begin{cases} 1 & n = 0 \\ a^{n/2} \times a^{n/2} & n \text{ is even} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & n \text{ is odd} \end{cases}$$

Recursive Programming (Exponentiation Efficient)

```
def exp(a, n):  
    if n == 0:  
        return 1    #base case  
    elif n % 2 == 0:  
        return exp(a, n/2) * exp(a, n/2)    #recursive part  
    else:  
        return exp(a, (n-1)/2) * exp(a, (n-1)/2) * a    #recursive part
```

Recursive Programming (Exponentiation Efficient)

```
def exp(a, n):  
    if n == 0:  
        return 1    #base case  
    elif n % 2 == 0:  
        temp = exp(a, n/2)    #recursive part  
        return temp * temp  
    else:  
        temp = exp(a, (n-1)/2)    #recursive part  
        return temp * temp * a
```


Recursive Programming (Problems)

- **Inefficient Recursion**
- **Space for Activation Frames**
- **Infinite Recursion**

Recursive Programming (Advanced)

- **Top-Down Approach**
- **Redundancies**
- **Memoization**

Recursive Programming (Steps)

- 1. Write down the recursion,**
- 2. Implement the recursive solution,**
- 3. Memoize it,**
- 4. Transform into an iterative solution, and finally**
- 5. Make further improvements.**

Recursive Programming (Fibonacci : Step 1)

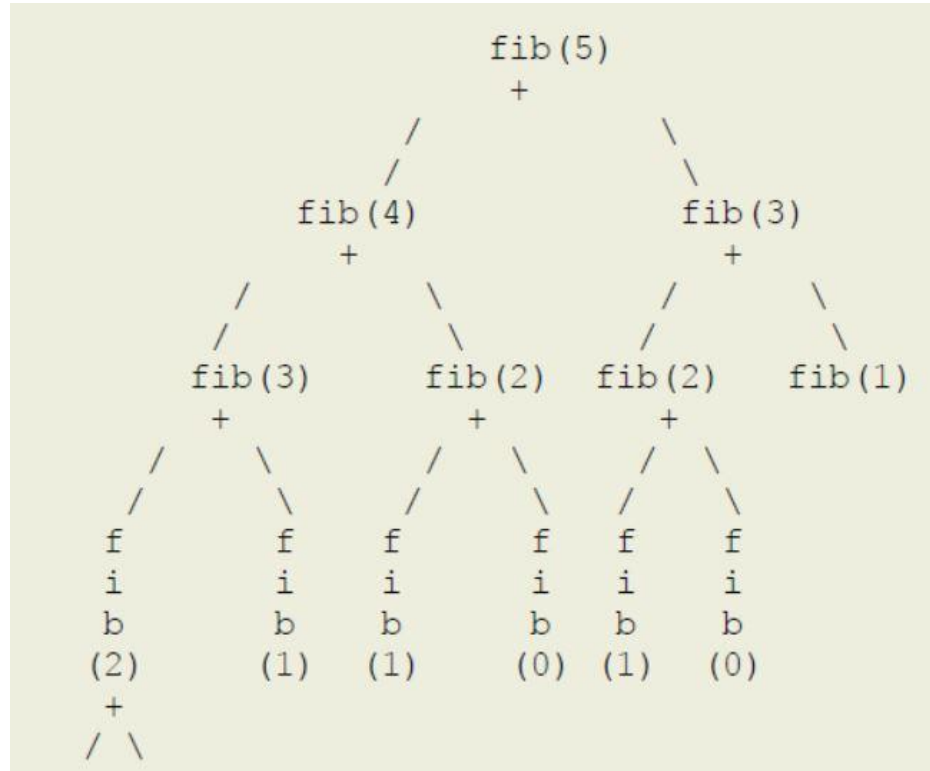
```
fib(n) = 
$$\begin{array}{ll} n & \text{if } n < 2 \\ \text{fib}(n-1) + \text{fib}(n-2) & n \geq 2 \end{array}$$

```

Recursive Programming (Fibonacci : Step 2)

```
def fib(n):  
    if n < 2:  
        return n    #base case  
    else  
        return fib(n-1) + fib(n-2)    #recursive part
```

Recursive Programming (Fibonacci : Step 3)



Recursive Programming (Fibonacci : Step 3)

memoization trades space for time

Linear Array. Size : $n+1$

Recursive Programming (Fibonacci : Step 3)

```
def M_fib(n):  
    # Assume that we have some "global" array (also called a table)  
    # F with n+1 capacity. Why can F not be a local variable?  
  
    if n < 2:  
        return n    #base case  
    else:  
  
        #Compute (and save) if it's not already computed  
  
        if (F[n] is empty)  # <<<< NOTE PSEUDOCODE!  
            F[n] = M_fib(n-1) + M_fib(n-2)    #recursive part  
  
        # Now just return the computed (and saved) value.  
        return F[n]
```


Recursive Programming (Fibonacci : Step 3)

```
def fib(n):  
    # Create and initialize the table.  
    F = [-1]*(n+1)  
  
    #Now we can call M-Fib with this extra parameter "F".  
    return M_fib(n, F)  
  
def M_fib(n, F):  
    #The table "F" is being passed as a parameter  
  
    if n < 2:  
        return n    #base case  
  
    else:  
        #Compute (and save) if it's not already computed.  
        if F[n] == -1:  
            F[n] = M_fib(n-1, F) + M_fib(n-2, F)    #recursive part  
  
        #Now just return the computed (and saved) value.  
        return F[n]
```

Recursive Programming (Fibonacci : Step 4)

```
def fib(n):  
    F=[None]*(n+1)    #The table to store computed values  
    F[0]=0             #The base case for n = 0  
    F[1]=1             #The base case for n = 1  
  
    for i in range(2,n+1):  
        F[i]=F[i-1]+F[i-2]  
    return F[n]
```

Recursive Programming (Fibonacci : Step 5)

```
def fib(n):  
    f_2 = 0      #The (n-2)th value  
    f_1 = 1      #The (n-1)th value  
    f = n  
    #The result f is initialized to n (why? So that it works when n is 0 or 1).  
  
    for i in range(2, n+1):  
        f = f_1 + f_2  
  
        #Now update the f_1 and f_2 for the next iteration (if any).  
        f_2 = f_1  
        f_1 = f  
  
    return f
```